

Age of Information Minimization on Graphs with Graph-Aware Reinforcement Learning (Final Solution and Extended Report)

Roman Iashchenko*, Manyou Ma (Instructor)[†] *HSE University, Faculty of Computer Science, Moscow, Russia
Email: roma.yashchenko3@gmail.com [†]School of Artificial Intelligence, Shenzhen Technology University, Shenzhen, China
Email: mamanyou@sztu.edu.cn

Abstract—We study Age of Information (AoI) minimization in a graph-based status update system under a one-action-per-step constraint. We implement a simulator with stochastic refresh dynamics, compare classical heuristics, and develop graph-aware deep reinforcement learning via a GNN-based DQN (GNN-DQN). We evaluate across multiple graph families (line, star, Erdos–Renyi), analyze generalization gaps, and propose a final solution emphasizing robustness across unseen topologies. To satisfy an extended-review format, we include implementation details and code excerpts in the appendix.

Index Terms—Age of Information, Reinforcement Learning, Deep Q-Network, Graph Neural Networks, Generalization, Networked Control

I. INTRODUCTION

Age of Information (AoI) measures data freshness: the time since the most recently received update was generated. AoI optimization appears in IoT monitoring, telemetry, caching, and distributed systems. In graph-structured systems, refreshing one node can indirectly benefit other nodes via diffusion/gossip-like mechanisms. As a result, the optimal policy depends on the graph topology, motivating graph-aware methods.

II. PROBLEM SETUP AND MDP

Let $G = (V, E)$ be an undirected graph with $|V| = N$ nodes. Each node i has AoI $a_i(t) \in \mathbb{R}_{\geq 0}$. At each discrete step t , the agent chooses one node $u(t) \in \{1, \dots, N\}$ to refresh. A refresh succeeds with probability p ; on success, the chosen node is reset to 0 (and optionally neighbors can be reset stochastically, depending on the simulator variant). We use the per-step reward:

$$r(t) = -\frac{1}{N} \sum_{i=1}^N a_i(t), \quad (1)$$

so maximizing return corresponds to minimizing mean AoI. Episodes have a fixed horizon T .

A. Graph Families

We evaluate on three representative graph families (all with $N = 10$): (i) line (path), (ii) star, (iii) Erdos–Renyi $G(N, p_e)$. The ER family is used both as a training distribution (for generalized training) and as an unseen test set (“erdos_new”).

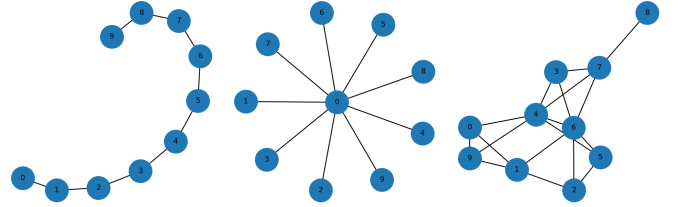


Fig. 1: Illustrations of evaluated graph families (line, star, Erdos–Renyi).

III. SIMULATOR

A. Environment Dynamics

At each step, ages increase by $+1$ for all nodes, then a refresh attempt is applied to the selected node (and optionally neighbors). The simulator follows a Gym-style interface with `reset()`, `step(action)` and trajectory generation utilities. Evaluation reports the mean AoI averaged over time, then averaged over multiple episodes and seeds.

IV. BASELINES

We compare against three classical heuristics:

- **Random**: select a node uniformly at random.
- **Greedy (stale-first)**: select $\arg \max_i a_i(t)$.
- **Degree-weighted**: select $\arg \max_i (\deg(i) \cdot a_i(t))$.

V. RL METHODS

A. DQN (vector state)

A standard DQN uses the AoI vector as state and outputs $Q(s, i)$ for each node action. This can work on a fixed graph but does not explicitly encode relational structure.

B. GNN-DQN (graph-structured value function)

To incorporate topology, we use a message-passing encoder (GCN-style) over node features. Each node receives features such as current AoI and degree-derived signals; the network outputs a per-node Q-value. The greedy action $\arg \max_i Q(s, i)$ defines the policy.

TABLE I: Key simulation and training parameters

Parameter	Value
Nodes N	10
Episode horizon T	50
Success probability p	0.8
Discount γ	0.99
Learning rate	1×10^{-3}
Hidden size	64
Batch size	64
Replay buffer size	20000
Warmup	500 steps
Target update	every 200 steps
Exploration	ϵ -greedy (multiplicative decay)

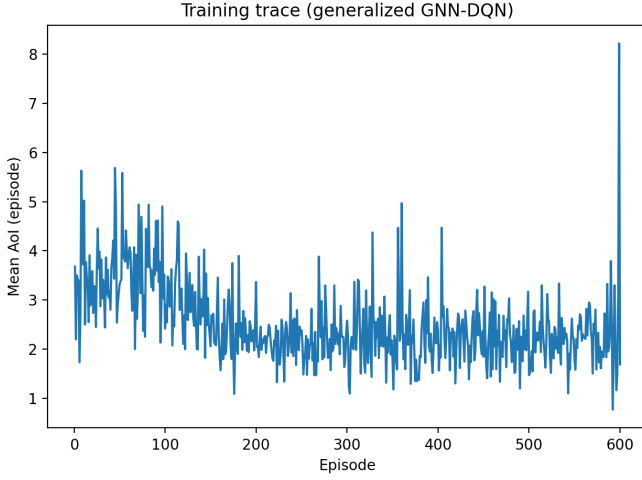


Fig. 2: Training trace (generalized GNN-DQN): episodic mean AoI vs. episode index.

C. Training Regimes

We distinguish two regimes:

- **Fixed-graph training:** train and evaluate on a single graph instance/type.
- **Generalized training:** re-sample a new Erdos–Renyi graph each episode from a distribution and evaluate on unseen graphs (line/star/new ER).

VI. EXPERIMENTAL SETUP

Table I summarizes the main hyperparameters used in the experiments.

VII. RESULTS

A. Training Trace

Figure 2 shows episodic mean AoI during generalized training (lower is better).

B. Final Performance Across Families

Figure 3 aggregates final performance (mean AoI; lower is better) across graph families.

C. Numerical Snapshot

Table II reports one concrete evaluation snapshot (mean AoI; lower is better).

TABLE II: Example numbers (mean AoI; lower is better)

Policy	Line	Star	Erdos_new
Random	4.182	5.460	3.234
Greedy	2.823	4.700	2.494
Degree-weighted	2.733	1.765	2.089
GNN-DQN (final)	3.920	0.579	7.317

D. Policy Diagnostics

Action histograms help detect policy collapse (e.g., repeatedly selecting the same node regardless of state).

VIII. DISCUSSION

A. What worked well

Degree-weighted is consistently strong on random graphs (ER family), suggesting degree is a robust proxy for propagation influence in the simplified dynamics. GNN-DQN can achieve excellent performance on star graphs, often learning to refresh the hub node, which is close to optimal under diffusion-like assumptions.

B. Generalization gap

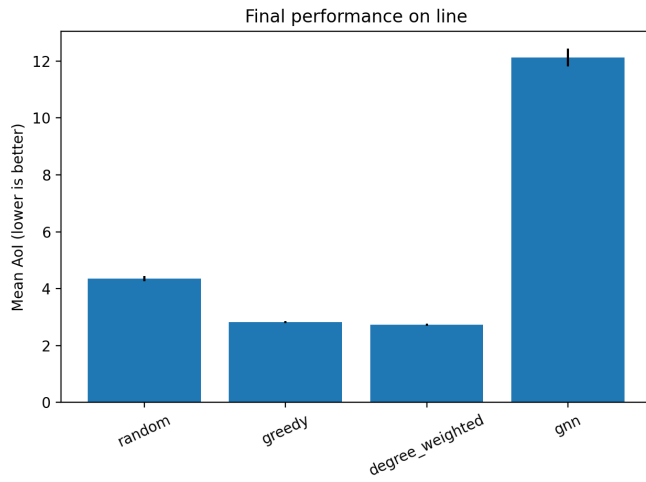
The main difficulty is robust generalization to unseen random graphs: the learned policy can overfit to training distribution artifacts or collapse to a small subset of actions. This is reflected in action histograms and high variance on erdos_new.

IX. FINAL RECOMMENDED SOLUTION (ROBUSTNESS-FIRST)

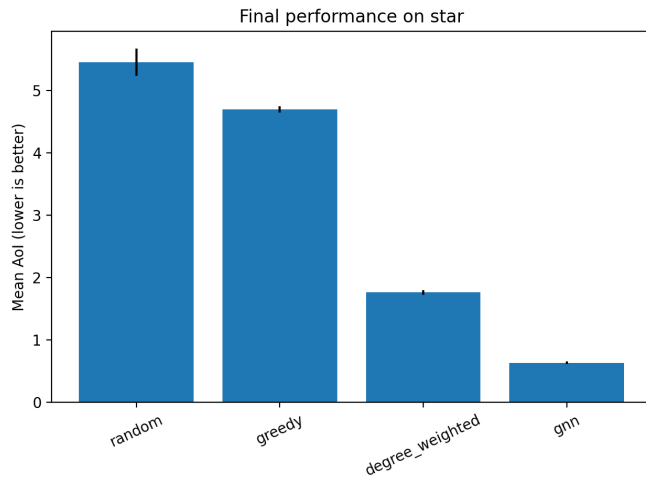
Given the empirical outcomes, we recommend a robustness-first final policy: use degree-weighted as the default on unknown graphs, optionally combined with a simple structure detector: if the graph is star-like (very high max degree), refresh the hub; otherwise use degree-weighted. This hybrid keeps performance strong on star-like structures while avoiding the worst-case collapses observed for learned policies on unseen ER graphs.

X. CONCLUSION

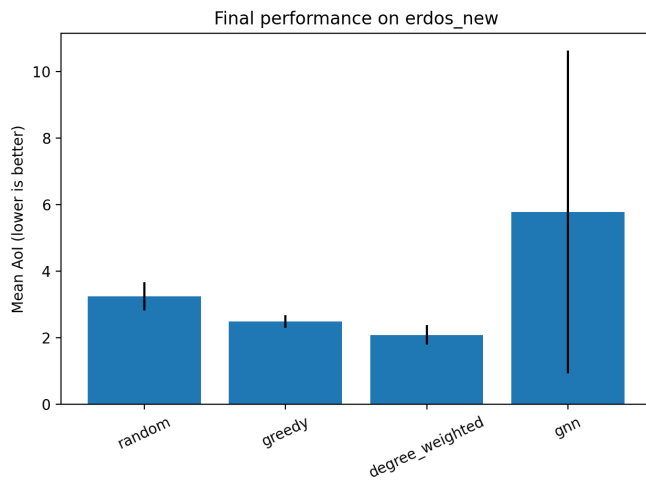
We implemented an AoI simulator on graphs, validated classical baselines, and trained graph-aware RL agents. While GNN-DQN can be very strong on certain families (e.g., star), degree-weighted remains the most robust across unseen random graphs. The final recommended solution prioritizes cross-topology reliability.



(a) Line graphs.

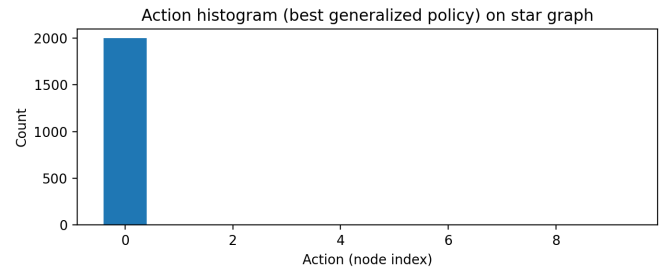


(b) Star graphs.

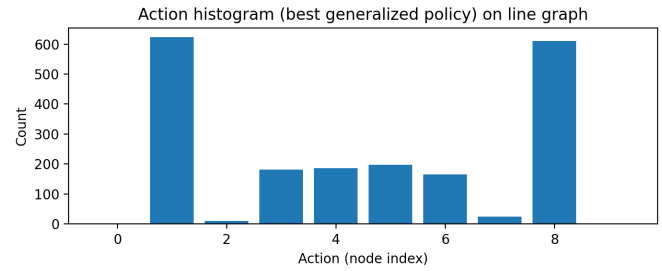


(c) Unseen ER graphs (erdos_new).

Fig. 3: Final mean AoI across graph families (lower is better).



(a) Star graph.



(b) Line graph.

Fig. 4: Action histograms for the learned policy.

APPENDIX A IMPLEMENTATION EXCERPTS (FOR EXTENDED REVIEW)

A. Environment Step (schematic)

```
def step(action):
    ages = ages + 1
    if bernoulli(p):
        ages[action] = 0
        for nb in neighbors[action]:
            if bernoulli(p):
                ages[nb] = 0
    reward = -ages.mean()
    return ages, reward, done, info
```

B. Degree-weighted Baseline

```
def degree_weighted_policy(ages, deg):
    score = deg * ages
    return int(score.argmax())
```

C. GNN-DQN Forward Pass (schematic)

```
def forward(data):
    x, edge_index = data.x, data.edge_index
    h = gcn1(x, edge_index).relu()
    h = gcn2(h, edge_index).relu()
    q = head(h).squeeze(-1)
    return q
```

REFERENCES

- [1] S. Kaul, M. Gruteser, V. Rai, and J. Kenney, “Real-time status: How often should one update?,” in *Proc. IEEE INFOCOM*, 2012.
- [2] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, and S. Ulukus, “Age of information: An introduction and survey,” *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 5, pp. 1183–1210, 2021.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [4] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [5] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. ICLR*, 2017.